

Redis + Node.js

การเพิ่มประสิทธิภาพระบบด้วย Caching & Performance

Production Ready Guide

สำหรับ Backend Developer | Node.js + Express.js | Beginner → Intermediate

Redis คืออะไร?



In-Memory Data Store

Remote Dictionary Server

- ▶ เก็บข้อมูลใน RAM ไม่ใช่ Disk
- ▶ ความเร็วระดับ microseconds
- ▶ รองรับ Data Structure หลากหลาย
- ▶ Open Source & Production Ready

ทำไมถึงเร็ว?



RAM Access: ~100 ns



Disk Access: ~10 ms

เร็วกว่า 100,000 เท่า!

Redis ใช้ทำอะไรได้บ้าง?



Cache

ลดโหลด Database
เพิ่มความเร็ว response



Session Store

จัดการ session
รองรับ scaling



Queue / Pub-Sub

จัดคิวงาน
real-time messaging



Rate Limiting

ป้องกัน spam
จำกัด request

Redis ไม่ใช่แค่ cache — เป็น multi-purpose in-memory data platform

ปัญหาที่ Redis แก้ได้



Database ช้า

Query ช้าๆ ที่ใช้เวลานาน
ผู้ใช้ต้องรอนาน



Query ช้าซ้อน

ข้อมูลเดิมถูก query
หลายร้อยครั้งต่อวินาที



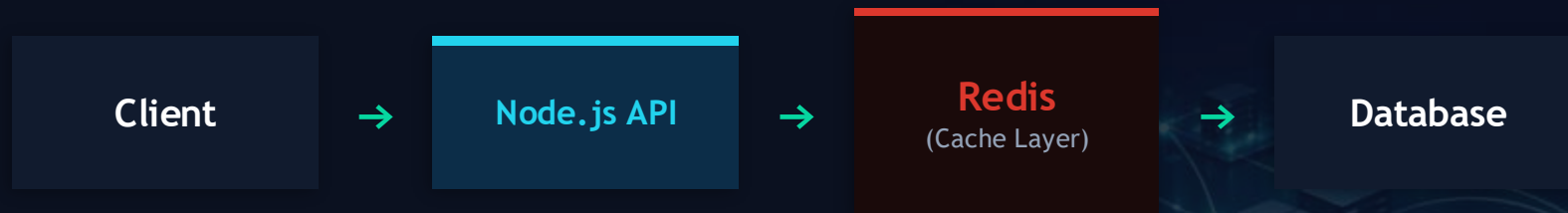
Load สูง

Server รับโหลดไม่ไหว
CPU/Memory พุง



Redis เป็นคำตอบ → Cache ข้อมูลที่ถูกเรียกบ่อย ลดโหลด DB ได้ 80-90%

ภาพรวม System Architecture



Flow: Client ส่ง request → API ตรวจสอบ cache → ถ้ามีใน Redis ส่งกลับทันที → ถ้าไม่มี query DB แล้วเก็บ cache

< 1ms

Response Time

100K+ ops/sec

Throughput

ลดลง 80-90%

DB Load

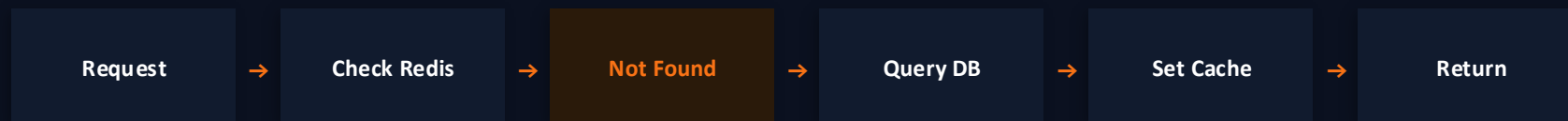
Cache Flow: Hit vs Miss

Cache HIT



⚡ เร็วมาก — ไม่ต้อง query Database

Cache MISS



ช้ากว่า Hit — แต่ครั้งต่อไปจะเป็น Hit แทน

ติดตั้ง Redis



Local Installation

```
# macOS
brew install redis
brew services start redis

# Ubuntu/Debian
sudo apt install redis-server
sudo systemctl start redis
```



Docker (แนะนำ)

```
# ดึง Redis image
docker pull redis

# รัน Redis container
docker run -d \
  --name my-redis \
  -p 6379:6379 \
  redis
```

ติดตั้ง Redis Client ใน Node.js

 ใช้ package 'redis' ซึ่งเป็น official client

```
# ติดตั้ง redis client  
npm install redis
```

```
# หรือใช้ yarn  
yarn add redis
```

Packages อื่นที่นิยม:

redis

Official — แนะนำ

ioredis

Feature-rich, Cluster support

connect-redis

Session store สำหรับ Express

การเชื่อมต่อ Redis กับ Node.js

```
// นำเข้า redis client
import { createClient } from "redis";

// สร้าง client instance
const client = createClient();

// จัดการ error
client.on("error", (err) => {
  console.log("Redis Client Error:", err);
});

// เชื่อมต่อ Redis server
await client.connect();

console.log("เชื่อมต่อ Redis สำเร็จ!");
```

หมายเหตุ

- ▶ ค่า default: localhost:6379
- ▶ รองรับ async/await
- ▶ ควร handle error เสมอ
- ▶ ใช้ environment variables สำหรับ production

คำสั่งพื้นฐาน: SET / GET

```
// SET - เก็บข้อมูล
await client.set("username", "somchai");
await client.set("score", "100");

// GET - ดึงข้อมูล
const name = await client.get("username");
console.log(name); // "somchai"

const score = await client.get("score");
console.log(score); // "100"

// เก็บ Object (ต้อง stringify)
const user = { name: "สมชาย", age: 25 };
await client.set("user:1", JSON.stringify(user));
const data = JSON.parse(await client.get("user:1"));
```

TTL – Time To Live



กำหนดอายุของ cache — หมดเวลาแล้วลบอัตโนมัติ

```
// SET พร้อม TTL (หน่วยเป็นวินาที)
await client.set("session", "abc123", { EX: 3600 });
// → หมดอายุใน 1 ชั่วโมง

// SET พร้อม TTL (หน่วยเป็น milliseconds)
await client.set("otp", "891234", { PX: 300000 });
// → หมดอายุใน 5 นาที

// เช็คเวลาที่เหลือ
const ttl = await client.ttl("session");
console.log(ttl); // จำนวนวินาทีที่เหลือ
```

แนะนำค่า TTL

Session	24 ชม.
API Cache	1-5 นาที
OTP	5 นาที
Dashboard	30 วิ.
Static Data	1 ชม.

DELETE — ลบข้อมูลจาก Cache

```
// ลบ key เดียว
await client.del("username");

// ลบหลาย key พร้อมกัน
await client.del(["user:1", "user:2", "user:3"]);

// ตรวจสอบว่า key มีอยู่หรือไม่
const exists = await client.exists("username");
console.log(exists); // 0 = ไม่มี, 1 = มี

// ลบทั้งหมด (ระวัง! ใช้เฉพาะ dev)
await client.flushAll();
```



ข้อควรระวัง: อย่าใช้ flushAll ใน production — จะลบข้อมูลทั้งหมด!

Cache Strategy

Cache Aside (Lazy Loading)

นิยมที่สุด

- ▶ App ควบคุม cache เอง
- ▶ เก็บ cache เมื่อ miss เท่านั้น
- ▶ เหมาะกับ read-heavy workload
- ▶ ง่ายต่อการ implement

Read-Through

- ▶ Cache layer จัดการเอง
- ▶ App ไม่ต้องรู้ว่ามี cache
- ▶ ซับซ้อนกว่า แต่ clean
- ▶ ใช้ library/middleware ช่วย

Cache Aside Flow

1 Request เข้ามา

2 Check Redis

3a HIT → Return ทันที

3b MISS → Query Database

4 Set Cache + TTL

5 Return Response

Key: ข้อมูลจะถูก cache หลัง miss ครั้งแรก → ครั้งต่อไปจะเป็น hit เสมอ (จนกว่า TTL หมด)

ตัวอย่าง: Express API + Redis Cache

```
app.get("/users", async (req, res) => {  
  // 1. เช็ค cache ก่อน  
  const cache = await client.get("users");  
  if (cache) {  
    return res.json(JSON.parse(cache)); // 2. HIT → ส่งกลับเลย  
  }  
  // 3. MISS → query จาก database  
  const users = await prisma.user.findMany();  
  // 4. เก็บผลลัพธ์ลง cache (TTL 60 วินาที)  
  await client.set("users", JSON.stringify(users), { EX: 60 });  
  res.json(users); // 5. ส่ง response  
});
```



Pattern นี้ใช้ได้กับทุก GET endpoint — เปลี่ยนแค่ key, query, และ TTL

ปัญหา Cache: Stale Data



ข้อมูลใน Cache เก่ากว่าข้อมูลจริงใน Database

T1

User A อ่านข้อมูล
→ Cache ถูกสร้าง

T2

User B แก้ไขข้อมูล
→ DB เปลี่ยน

T3

User C อ่านข้อมูล
→ ได้ข้อมูลเก่าจาก Cache!

นี่คือปัญหา *Stale Data* — ผู้ใช้เห็นข้อมูลที่ไม่ตรงกับความเป็นจริง

Cache Invalidation — วิธีแก้ปัญห

วิธีที่ 1: ใช้ TTL

- ▶ กำหนดเวลาหมดอายุ cache
- ▶ ข้อมูลจะ refresh อัตโนมัติ
- ▶ เหมาะกับ data ที่ไม่ต้อง real-time

วิธีที่ 2: ลบ Cache เมื่อ Update

- ▶ ลบ cache เมื่อมีการแก้ไขข้อมูล
- ▶ ข้อมูลจะ fresh เสมอ
- ▶ เหมาะกับ data ที่ต้อง real-time

```
// ลบ cache เมื่อมีการ update ข้อมูล
app.put("/users/:id", async (req, res) => {
  await prisma.user.update({ ... });

  // ลบ cache ที่เกี่ยวข้อง
  await client.del("users");
```

Rate Limiting ด้วย Redis



ป้องกัน spam / brute-force โดยจำกัดจำนวน request ต่อเวลา

```
// Rate Limiter – จำกัด 100 requests ต่อหน้าที่
async function rateLimit(ip) {
  const key = `rate:${ip}`;
  const current = await client.incr(key);

  if (current === 1) {
    // ตั้ง TTL เมื่อเป็น request แรก
    await client.expire(key, 60);
  }

  if (current > 100) {
    return false; // ❌ เกิน ลิมิต
  }

  return true; // ✅ ผ่าน
```

Session Store ด้วย Redis

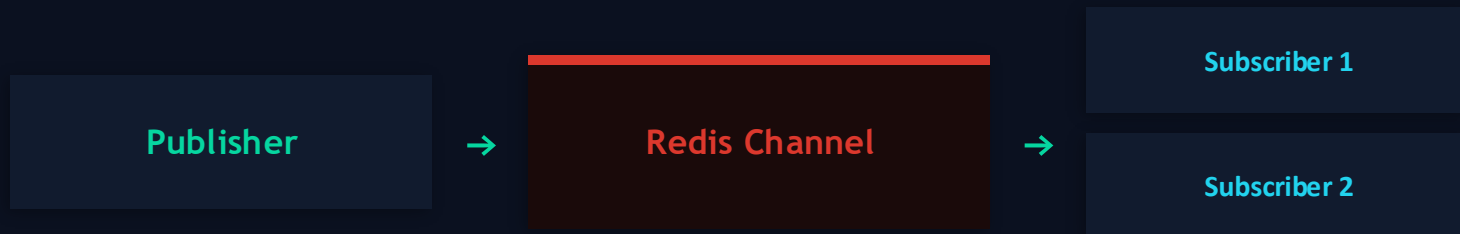


เก็บ session ใน Redis แทน memory — รองรับ scaling หลาย server

```
import session from "express-session";
import RedisStore from "connect-redis";

// สร้าง Redis session store
app.use(session({
  store: new RedisStore({ client }),
  secret: "my-secret-key",
  resave: false,
  saveUninitialized: false,
  cookie: {
    maxAge: 86400000 // 24 ชั่วโมง
  }
}));
```

Redis Pub/Sub – Real-time Communication



```
// Publisher – ส่งข้อความ
await client.publish("notifications", JSON.stringify({
  type: "order", message: "มีออเดอร์ใหม่!"
}));
```

```
// Subscriber – รับข้อความ
const subscriber = client.duplicate();
await subscriber.connect();
await subscriber.subscribe("notifications", (msg) => {
  const data = JSON.parse(msg);
```

```
  console.log("ได้รับ:", data.message);
```

```
});
```

ทำไม Redis ถึงเร็วมาก?



In-Memory Storage

ข้อมูลอยู่ใน RAM ทั้งหมด
ไม่ต้องอ่านจาก Disk



Single-Threaded Event Loop

ไม่มี context switching
ไม่มี lock contention



Optimized Data Structures

ออกแบบมาเพื่อความเร็ว
Hash, Sorted Set, etc.



Non-blocking I/O

Multiplexing connections
รองรับ concurrent สูง



Redis Best Practices



Key Naming Convention

ใช้ pattern: entity:id:field

เช่น user:123:profile



ตั้ง TTL เสมอ

ป้องกัน memory leak

ข้อมูลเก่าจะถูกลบอัตโนมัติ



อย่า Cache ข้อมูลใหญ่

จำกัดขนาด value

ใช้ compression ถ้าจำเป็น



Monitor Memory

ตรวจสอบ memory usage

ตั้ง maxmemory policy

Error Handling ใน Production

```
// จัดการ error ของ Redis client
client.on("error", (err) => {
  console.error("Redis Error:", err);
  // ส่ง alert ไปยัง monitoring system
});

// Graceful degradation – ถ้า Redis ล่ม ให้ fallback
async function getWithFallback(key, fetchFn) {
  try {
    const cached = await client.get(key);
    if (cached) return JSON.parse(cached);
  } catch (err) {
    console.warn("Redis unavailable, fallback to DB");
  }
}
```

```
// Fallback: query จาก DB โดยตรง
return await fetchFn();
```

Scaling Redis



Redis Sentinel

High Availability
Automatic failover
Monitoring



Redis Cluster

Data sharding
Horizontal scaling
Multi-node



Read Replicas

แยก read/write
ลดโหลด master
เพิ่ม throughput



เริ่มต้นด้วย Single Instance → Sentinel → Cluster ตามขนาดของระบบ

Redis Security



ปกป้อง Redis Server ใน Production

ตั้ง Password

requirepass ใน redis.conf

```
const client = createClient({ password: 'secret' });
```

Bind IP

จำกัดการเข้าถึงจาก IP ที่อนุญาต

```
bind 127.0.0.1 10.0.0.1
```

Firewall

เปิด port 6379 จากภายนอก

```
ufw allow from 10.0.0.0/24 to any port 6379
```

TLS/SSL

เข้ารหัสการเชื่อมต่อ

```
const client = createClient({ socket: { tls: true } });
```

Redis + Prisma ORM

```
// Helper: Cache wrapper สำหรับ Prisma queries
async function cachedQuery(key, ttl, queryFn) {
  // ดึงจาก cache ก่อน
  const cached = await client.get(key);
  if (cached) return JSON.parse(cached);

  // ถ้าไม่มี → query DB ผ่าน Prisma
  const result = await queryFn();

  // เก็บลง cache
  await client.set(key, JSON.stringify(result), { EX: ttl });
  return result;
}

// ใช้งาน
const users = await cachedQuery("users:all", 300, () =>
```

Use Cases ในงานจริง



Dashboard API

Cache ข้อมูล analytics

ที่ query หนักๆ

TTL: 30 วินาที



API ขอดนิยมน

Cache product listing

ลดโหลด DB ได้ 90%

TTL: 5 นาที



User Profile

Cache user data

ที่ถูกเรียกบ่อย

TTL: 10 นาที



Search Results

Cache ผล search

ลดเวลา response

TTL: 1 นาที

สรุป



Redis = In-Memory Data Store ที่เร็วระดับ microseconds



Caching = หัวใจของ Performance — ลดโหลด DB ได้ 80-90%



ตั้ง TTL เสมอ + Invalidate cache เมื่อ data เปลี่ยน



ใช้ Rate Limiting & Session Store เพื่อ security



Error Handling + Graceful Degradation สำหรับ production

นำ Redis ไป optimize ระบบจริงของคุณได้เลย!



Q & A

ถาม-ตอบ & แลกเปลี่ยนความรู้

ขอบคุณที่ร่วมเรียนรู้!